

A Service-Oriented Modular Architecture for Domain-Adaptive Retrieval-Augmented Generation

Sruthi Satyavarapu, Pramod Gupta, Mohammed Fahad

Department of Computer Science, University of the Pacific

Stockton, CA - USA

s_satyavarapu@u.pacific.edu, pgupta@pacific.edu, m_lun18@u.pacific.edu

Abstract—Large Language Models (LLMs) combined with Retrieval-Augmented Generation (RAG) have emerged as a practical approach for building knowledge-grounded conversational systems. As these systems are increasingly deployed in data-centric environments, the need for systems that support multi-domain adaptability becomes critical. However, deploying RAG systems across heterogeneous domains remains challenging, as retrieval performance and response quality depend heavily on the choice of embedding models, retrieval strategies, vector stores and language model backends. Existing RAG toolkits either focus on algorithmic benchmarking with limited support for stateful, multi-user deployment or on application development with limited support for component-level experimentation and observability, forcing practitioners to choose between research flexibility and production readiness.

We propose a deployment-oriented, modular RAG architecture that enables controlled experimentation across domains while incorporating stateful system services. Pipeline components are independently configurable and extensible as the RAG ecosystem evolves. A dedicated system services layer provides persistent storage, session-aware interaction and role-based access control. In addition, integrated latency instrumentation enables fine-grained performance profiling alongside quality evaluation. We implement and evaluate the system across a Sanskrit philosophical corpus and a biomedical abstract corpus to investigate RAG behavior across heterogeneous domains. Experimental results show that identical configurations can exhibit substantially different retrieval and generation characteristics across domains. Multi-turn evaluation demonstrates that session-aware interaction preserves conversational relevance across turns, while latency profiling reveals non-obvious performance interactions between pipeline stages. Collectively, these findings highlight the importance of modularity, statefulness and observability in the design of RAG systems intended for both deployment and cross-domain research.

Index Terms—Retrieval-Augmented Generation (RAG), Conversational AI, Domain-Adaptive AI Systems, Service-Oriented AI Systems, RAG Evaluation

I. INTRODUCTION

Large Language Models (LLMs) have recently transformed a wide range of applications, including conversational agents, automated assistants and knowledge-driven data services. With the proliferation of large-scale data, LLMs are being actively integrated into data-intensive systems for a myriad of tasks. Recent work has highlighted the growing integration of LLMs with data-centric systems, where models operate over large-scale structured and unstructured data to support knowledge discovery and information access [1]. This trend is also reflected in industry platforms such as Databricks [2] and

Snowflake [3] which have begun integrating LLM capabilities directly into data processing and analytics workflows. However, despite their strong generative capabilities, LLMs remain prone to hallucinations [4], knowledge temporality and other limitations [5] limiting their reliability in applications where factual correctness and data grounding are crucial.

Retrieval-Augmented Generation (RAG) introduced by Lewis et al. [6], addresses this limitation by integrating external knowledge sources into the generation process. In a typical RAG pipeline, a user query is used to identify relevant information from an external knowledge base, often from a vector database using embedded representations. The retrieved content is often refined through additional processing steps such as query rewriting [7], re-ranking [8] or context compression [9] before being combined with the original query and passed to the LLM. This enables the LLM to generate responses grounded in relevant retrieved context. By decoupling knowledge storage from model parameters, RAG enables LLMs to operate over dynamic and domain specific data sources, making it suitable for a wide range of real-world applications like domain specific chatbots, enterprise assistants [10] and code completion systems [11], [12].

Recent surveys [13], [14] demonstrate that RAG has been widely applied across diverse domains such as healthcare, finance, education and legal systems. However, these systems are designed around domain specific datasets and workflows resulting in limited reusability and poor adaptability. Furthermore, existing RAG pipelines are often tightly coupled, with fixed combinations of retrievers, embedding models and generation strategies making it difficult for engineers or researchers to experiment and systematically evaluate individual components. The complexity of such systems is further amplified by interactions between multiple components, where changes in a single component such as the embedding model or retrieval strategy can significantly affect both response quality and end-to-end latency. As a result, identifying effective configurations across domains becomes a non-trivial task.

To address this gap, we propose a service-oriented RAG architecture that unifies three capabilities: configuration-driven pipeline modularity, deployment-ready system services and component-level observability. Pipeline components including embedding models, retrieval mechanisms, re-ranking, rewriting modules, vector databases and LLM backends are independently configurable and extensible, with new components

easily added as the RAG ecosystem evolves. A dedicated system services layer provides persistent storage, session-aware interaction and role-based access control, with built-in component-level latency instrumentation enabling fine-grained performance profiling. We implement this architecture as an end-to-end system with a centralized orchestration layer and evaluate it across two heterogeneous domains — a biomedical corpus and a Sanskrit philosophical corpus. The evaluation spans retrieval effectiveness, generation quality, session-aware multi-turn interaction and component-level latency instrumentation, demonstrating that RAG performance is highly dependent on both domain characteristics and component selection.

II. RELATED WORK

Retrieval-Augmented Generation (RAG) has gained significant attention as a method for improving factual grounding in large language models by utilizing external knowledge sources. Early work by Lewis et al. [6] introduced the RAG paradigm and subsequent research has explored various methodologies to improve on it.

A number of frameworks such as LangChain [15] and LlamaIndex [16] have emerged to support the development of RAG-based applications. These systems provide abstractions for integrating retrieval and generation components; however, they are primarily designed for application development and often involve tightly coupled workflows, limiting their suitability for controlled experimentation and systematic evaluation across domains. RAGLab [17] is one of the earliest papers to address the need for modular RAG systems, proposing a framework that emphasizes benchmarking and reproducibility but primarily focuses on comparison of different RAG algorithms.

Recent research such as Modular RAG [18] re-framed RAG systems as “LEGO-like” architectures, enabling decomposition into independent operators accommodating routing, branching and fusion and supporting beyond linear pipelines. Further extending these ideas, FlashRAG [19] provides a comprehensive, research-oriented toolkit that integrates multiple RAG components, benchmark datasets and pre-implemented algorithms within a unified modular framework. It emphasizes reproducibility and fair comparison by supporting diverse retrieval strategies, multi-modal scenarios and standardized evaluation pipelines. Similarly, UltraRAG [20] focuses on adaptive RAG system construction, providing an automated and modular framework for configuring and optimizing RAG pipelines across different tasks and datasets. Together, these approaches highlight the importance of modularity, flexibility and reproducibility in RAG systems. However, their primary focus remains on benchmarking, pipeline composition and automated configuration of RAG methods, with limited emphasis on system-level architectural concerns such as persistent state management, deployment-oriented design and support for domain-adaptive, multi-user environments.

In parallel, domain specific adaptations of retrieval-augmented generation have been explored to improve performance in specialized settings. Prior work has shown that RAG

systems built on general purpose corpora struggle to generalize to domain specific knowledge bases, necessitating tailored retrieval strategies and fine-tuning. For instance, approaches such as RAFT [21] propose fine-tuning to adapt large language models to domain specific corpora, while Siriwardhana et al. [10] explore joint retriever-generator training for improved domain transfer. Such studies indicate that adapting RAG systems across domains is non-trivial, as performance depends on multiple interacting components. Consequently, effective domain adaptation requires systematic exploration of configurations rather than a single fixed design. This highlights the need for architectures that support configuration-driven adaptations across heterogeneous domains without requiring system redesign.

Collectively, existing frameworks occupy two distinct positions: benchmarking-oriented systems such as FlashRAG and UltraRAG and application-oriented frameworks such as LangChain and LlamaIndex. Neither category supports the simultaneous need for stateful multi-user deployment and systematic cross-domain research instrumentation. The proposed architecture is designed to occupy this gap by providing production-level system capabilities — including persistent storage, session management and integrated latency instrumentation — alongside the component-level configurability necessary for controlled empirical research, serving as infrastructure for both deployment and discovery.

III. SERVICE-ORIENTED MODULAR RAG ARCHITECTURE

The proposed system is designed as a modular, service-oriented architecture that decomposes Retrieval-Augmented Generation (RAG) into configurable components. The architecture follows a design such that individual modules are implemented as independent services, enabling flexible composition, scalability and controlled experimentation across configurations. The architecture is guided by four primary design principles:

- **Separation of Concerns:** The system is organized into four distinct layers: interaction, orchestration, processing and system services. Each layer maintains clear functional boundaries, enabling modular integration.
- **Configuration-driven Modularity:** The architecture supports interchangeable components, including embedding models, retrieval strategies, prompt configurations, vector databases, persistent storage and LLM backends without significant architectural modifications.
- **Extensibility:** The architecture is designed to support integration of additional capabilities such as hybrid retrieval, knowledge graph-based retrieval and domain specific adaptations within a unified pipeline.
- **Observability:** A dedicated instrumentation layer captures component-level execution across each pipeline stage. Latency profiling is implemented natively, and the layer is extensible to broader logging and tracing as needed.

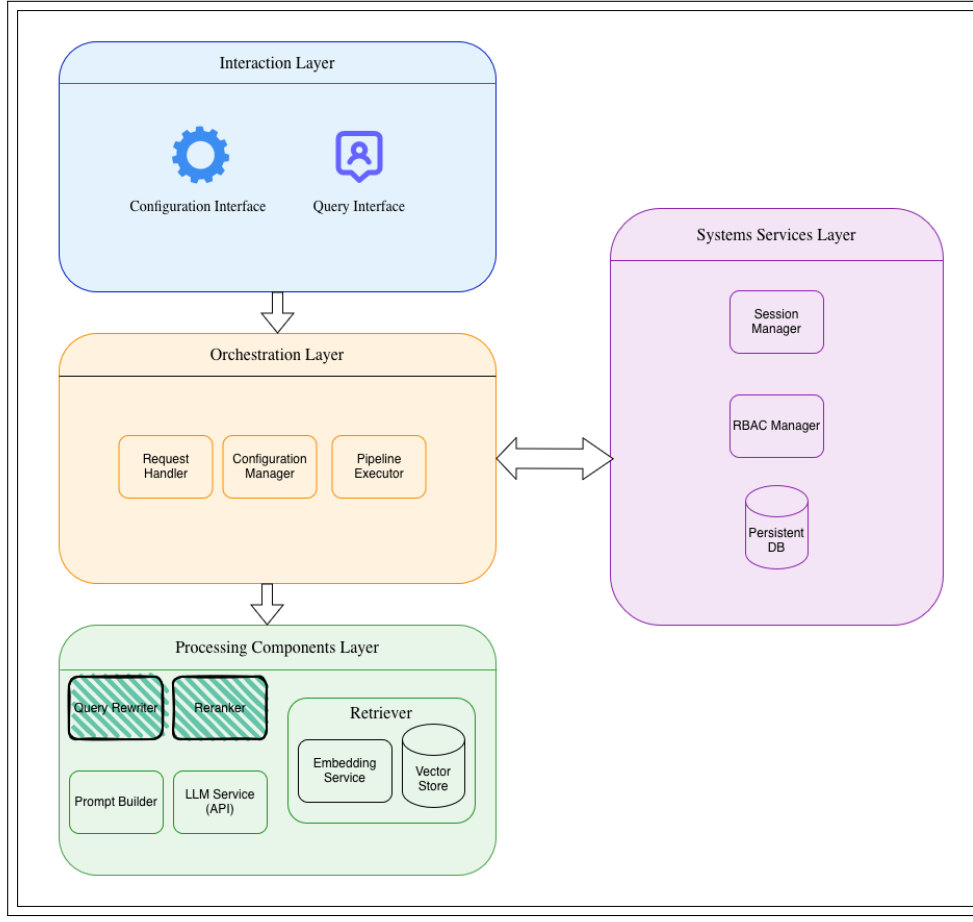


Fig. 1. Service-Oriented Modular RAG Architecture - Optional components are visually distinguished

A. Layered System Architecture

Figure 1 illustrates the proposed system architecture, organized into four distinct layers:

1) *Interaction Layer (Frontend Service)*: The interaction layer consists of two primary components: a query interface and a configuration interface. The query interface allows users to submit queries and receive generated responses, while the configuration interface enables users to specify system configurations, including embedding models, retrieval strategies, prompt templates, re-ranking and rewriting components, LLM backends and system level settings such as session-aware interaction. Both components communicate with the backend orchestration layer through a unified service interface, enabling dynamic configuration and controlled experimentation. Furthermore, the architecture is designed to be interface-agnostic, allowing users to integrate a custom frontend that interacts with the orchestration layer through the same service interface (API).

2) *Orchestration Layer*: The orchestration layer functions as the central coordinating component of the system, responsible for managing the execution of the RAG pipeline based on the selected configuration. It comprises three internal components: a Request Handler that receives incoming queries

from the Interaction Layer, a Configuration Manager that retrieves session context and active pipeline settings from the System Services Layer and a Pipeline Executor that invokes processing components in a structured pipeline based on the resolved configuration.

Components such as re-ranking, rewriting and LLM backends can be dynamically modified across queries, enabling rapid experimentation with different configurations without restart. The orchestration layer also manages interactions with supporting services, including vector stores, persistence layers and state management systems, and instruments each pipeline stage by recording start and end timestamps around every component invocation, capturing independent latency measurements without requiring component-level modifications.

The modular design of the orchestration layer further enables extensibility, allowing new components such as knowledge graph-based retrieval, hybrid search strategies or domain specific reasoning modules to be integrated without significant architectural changes.

3) *Processing Component Layer*: The processing components layer contains the core RAG functionality through a set of modular services including a Query Rewriter, a Retriever comprising an Embedding Service and Vector Store,

a Re-ranker, a Prompt Builder and an LLM Service that communicates with external large language model backends. Each component is designed as an independent module with well-defined interfaces and allows the orchestration layer to invoke it based on the selected configuration. Additional services can be integrated into this layer without modifying the orchestration logic, supporting extensibility as new retrieval or processing strategies emerge.

A key characteristic of this layer is the distinction between dynamically configurable and configuration-dependent components. Components such as query rewriting, re-ranking, prompt construction and LLM inference can be modified dynamically across queries. In contrast, components such as embedding models, vector stores and system-level services require re-initialization when configurations are modified.

4) *System Services Layer*: The system services layer provides the supporting infrastructure required for persistent storage, state management and controlled access to information. Unlike conventional RAG pipelines that operate in a largely stateless manner, this layer enables the system to function in multi-user, domain-sensitive environments by maintaining contextual and user-specific information across interactions.

A modular persistent storage component is used to manage user profiles, session history, system metadata and per-user pipeline configurations. Each user is associated with a unique identifier, enabling personalized query processing, context-aware retrieval and configuration-driven pipeline composition tailored to individual users. In addition, role-based access control (RBAC) mechanisms are incorporated to regulate access to both data and configuration dimensions — for example, administrative roles may modify infrastructure-level settings such as the vector store backend, while standard users may adjust query-time options including LLM selection or query rewriting. A conceptual view of the configuration model is shown below:

```
admin_tier:
  vector_store: chromadb
  embedding_model: bge-large-en

user_tier:
  llm_backend: llama-3-70b
  query_rewriting: enabled
  reranker: enabled
  prompt_template: medical_v2
```

This integration of configuration management with access control distinguishes the proposed architecture from frameworks where pipeline composition is governed by static configuration files. Session management is handled through a dedicated in-memory system, allowing efficient tracking of user interactions and preserving context across multi-turn queries. To address the challenge of growing interaction histories, the system incorporates context compression and summarization mechanisms. As conversation logs expand, summarization strategies are applied to retain relevant information while

reducing context size, enabling efficient downstream retrieval and response generation. The impact of session-aware retrieval on response quality is empirically evaluated in Section IV.

The architecture maintains modularity at this layer by allowing alternative storage backends or role-based controls to be integrated with minimal structural changes. Such modifications are configuration-dependent and typically require system re-initialization, consistent with other non-dynamic components of the system.

By unifying the four layers — interaction, orchestration, processing and system services — together with centralized observability, the architecture extends beyond isolated RAG pipelines to support deployment-ready, stateful and observable multi-user systems within a single framework.

IV. EVALUATION

To evaluate the proposed modular RAG architecture, we conducted controlled experiments across two heterogeneous domains: (i) A Sanskrit philosophical corpus derived from the Bhagavad Gita and (ii) a medical corpus constructed using biomedical abstracts derived from PubMedQA [22]. These domains were selected to capture the variation in structure and knowledge representation, enabling analysis of how different RAG components behave under diverse domains. The objective of the evaluation is not to optimize for a single best-performing configuration but to demonstrate how the modular framework supports systematic experimentation across multiple dimensions. In this paper, we evaluate four representative dimensions — retrieval effectiveness, end-to-end generation quality, session-aware multi-turn interaction and component-level pipeline latency. The modular design enables straightforward extension to additional evaluation dimensions.

A. Experimental Setup

1) *Domains and Corpora*: **Sanskrit Domain (Bhagavad Gita)**: The Sanskrit corpus consists of multiple chapters of an English translation of the Bhagavad Gita. The corpus was preprocessed and segmented into fixed-length chunks. The domain represents long-form philosophical discourse characterized by conceptual abstraction and recurring thematic elements. Queries were designed to require semantic retrieval of conceptually relevant passages.

Medical Domain (Biomedical Abstract Corpus): The medical corpus was constructed using queries and associated abstracts derived from the PubMedQA Dataset. In the original dataset, many queries are directly linked to a single source abstract, which may result in trivial one-to-one retrieval scenarios. To create a more realistic multi-document retrieval setting, selected topics were augmented with additional biomedical abstracts addressing the same disease or medical concept from independent sources. This augmentation ensures that related information may appear across multiple documents rather than being confined to a single abstract. Importantly, no textual content was modified; only corpus composition was expanded to distribute conceptually related information across documents.

For both domains:

- Documents were chunked into fixed-length segments of 512 tokens with 50 token overlap.
- Ground-truth relevant chunks were identified per query.
- Each domain included approximately 50 queries.
- All retrieval experiments used top-5 document retrieval ($k = 5$).

While the current evaluation comprises 50 queries per domain, the focus of this work is component-level and system-level sensitivity analysis rather than exhaustive statistical benchmarking. Broader evaluation across expanded query sets, additional domains and models is left for future work, with the current scope constrained by API costs.

2) *Evaluation Metrics*: Retrieval performance was evaluated independently of generation using standard ranking-based metrics that are commonly used in information retrieval and retrieval-augmented generation literature [23], [14].

Recall@5: Recall@5 measures the proportion of relevant chunks retrieved within the top 5 results. For a given query q , it is defined as:

$$\text{Recall@5}(q) = \frac{\# \text{ relevant chunks retrieved in top 5}}{\# \text{ total relevant chunks}} \quad (1)$$

The final Recall@5 score is computed as the average across all queries in the domain.

Mean Reciprocal Rank (MRR): MRR evaluates ranking quality by considering the position of the first relevant chunk in the retrieved list. For a query q , the reciprocal rank is defined as:

$$\text{RR}(q) = \frac{1}{\text{rank of first relevant chunk}} \quad (2)$$

MRR is then computed as the mean of reciprocal ranks across all queries.

End-to-end response quality was evaluated using RAGAS [24], an automated evaluation framework with Gemini 2.5 Flash [25] as judge model. The following three metrics were computed across two domains mentioned earlier and two model configurations i.e., LLaMA 3 70B [26] and GPT-OSS 120B [27].

Faithfulness: Degree to which the generated response is supported by the retrieved context.

Relevance: Extent to which the response directly addresses the user query.

Correctness: Measures the factual accuracy of the generated response against reference answers, capturing the overall correctness of the response relative to the ground truth.

All scores are normalized to the range $[0, 1]$ for consistency.

B. Retrieval Performance

Retrieval effectiveness was evaluated independently of generation to isolate the impact of embedding configurations. Table I summarizes Recall@5 and MRR across the two domains.

In the Sanskrit domain, MiniLM L6 v2 achieved higher Recall@5 and MRR compared to MPNet Base v2, indicating

TABLE I
RETRIEVAL PERFORMANCE ACROSS DOMAINS

Domain	Embedding	Recall@5	MRR
Bhagavad Gita	MiniLM L6 v2	0.821	0.744
Bhagavad Gita	MPNet Base v2	0.762	0.681
Medical	MiniLM L6 v2	0.881	0.810
Medical	MPNet Base v2	0.842	0.773

stronger semantic alignment for abstract philosophical queries. The lower MRR values relative to the medical domain reflect the conceptual and interpretive nature of Sanskrit passages, where relevant content may appear across multiple semantically related chunks.

In the medical domain, both embedding configurations demonstrated strong retrieval performance, with Recall@5 exceeding 0.84. The structured terminology and domain specific vocabulary likely contributed to improved ranking stability and higher MRR values.

These results demonstrate domain-sensitive retrieval behavior and validate the modular architecture’s ability to support controlled embedding experimentation without modification of orchestration logic or system structure.

C. End-to-End Response Quality

Table II summarizes the end-to-end response quality across domains and LLM backends under fixed retrieval settings using BGE Large [28] as the embedding model. BGE Large was selected for generation evaluation as a higher-capacity embedding model to ensure retrieval quality did not bottleneck generation assessment.

TABLE II
END-TO-END RESPONSE QUALITY ACROSS DOMAINS

Domain	LLM Backend	Faithfulness	Relevance	Correctness
Bhagavad Gita	LLaMA 3 70B	0.838	0.911	0.465
Bhagavad Gita	GPT-OSS 120B	0.816	0.864	0.429
Medical	LLaMA 3 70B	0.304	0.751	0.563
Medical	GPT-OSS 120B	0.168	0.839	0.539

The evaluation findings reveal a striking domain-dependent divergence in generation behavior. In the Sanskrit philosophical corpus, both LLaMA 3 70B and GPT-OSS 120B demonstrated high faithfulness scores (0.838 and 0.816 respectively), indicating strong grounding in retrieved context. In contrast, both models exhibited substantially lower faithfulness in the medical domain (0.304 and 0.168 respectively), suggesting that models are relying more heavily on parametric knowledge. Despite the low faithfulness, answer relevance remained high at 0.751 and 0.839 for both models. This indicates that the models are generating relevant responses but bypassing the retrieved context — a known failure mode in RAG systems where parametric knowledge supersedes retrieved context [29]. This conflict is particularly pronounced in domains like medicine, where models are heavily pre-trained on publicly available corpora resulting in strong parametric knowledge [30]. Notably, answer correctness is higher in the medical domain (0.563 and 0.539) compared to the Sanskrit domain

(0.465 and 0.429) despite substantially lower faithfulness, suggesting that parametric knowledge can sustain factual accuracy even when retrieved context is bypassed. This divergence across metrics highlights how the proposed framework enables systematic diagnosis of RAG failure modes across domains.

D. Multi-Turn Session Evaluation

To evaluate the session management capability of the proposed architecture, we conducted a controlled multi-turn experiment on the medical domain. Ten conversation chains of three turns each were constructed, totaling 30 queries. Each conversation chain was designed such that the follow-up questions contain implicit references to prior context — for example, a Turn 1 query asking “What is coloboma?” followed by Turn 2 asking “How many people are affected by it?” and Turn 3 “What are the genetic changes related to it?”, where the pronoun “it” requires session context to resolve correctly. All chains used the same configuration — embedding model BGE Large, external LLM GPT-OSS 120B and identical pipeline settings — and were evaluated under two conditions: without session, where each query is processed independently with no conversation history and with session, where the full conversation history is passed as context at each turn.

Table III presents the results averaged across the chains. As expected, Turn 1 relevance is comparable across both conditions, with minor difference attributable to LLM response variability across independent runs rather than session management effects. Without session, relevance drops drastically from Turn 2 onward — reaching near zero (0.000 and 0.062) scores as the model fails to resolve implicit references without prior context. With session, relevance remains consistently high (0.572 and 0.664) across all turns, demonstrating that the architecture’s session management layer effectively preserves conversational context and enables coherent multi-turn retrieval. This result empirically validates the System Services Layer’s session management capability as a meaningful contributor to response quality in conversational deployment scenarios.

TABLE III
MULTI-TURN SESSION EVALUATION – MEDICAL DOMAIN (SCORES AVERAGED PER TURN ACROSS 10 CONVERSATION CHAINS). ALL SCORES REFLECT RAGAS ANSWER RELEVANCE NORMALIZED TO [0, 1].

Condition	Turn 1	Turn 2	Turn 3
Without Session	0.903	0.000	0.062
With Session	0.892	0.572	0.664

E. Pipeline Latency Analysis

A core capability of the proposed architecture is its native instrumentation for component-level latency profiling. Unlike conventional RAG pipelines where latency is measured end-to-end, the modular design enables independent timing of each pipeline stage — embedding, retrieval, optional query rewriting and LLM inference — allowing practitioners to reason about latency tradeoffs alongside quality metrics within the same framework. Table IV presents pipeline latency across

configurations on the medical domain, averaged over 25 queries. Outliers exhibiting query rewriting latency exceeding 20 seconds were excluded as API-level anomalies.

Retrieval consistently contributes negligible latency across all configurations (0.08–0.21s), demonstrating that the vector store layer scales independently of embedding or LLM choice — a direct benefit of the modular separation of retrieval and inference layers. Embedding latency reflects model capacity, with BGE Large averaging 4.03–4.38s compared to 2.37–3.29s for MPNet Base v2. LLM inference constitutes the dominant latency component, with GPT-OSS 120B averaging 1.94–2.41s compared to 14.00–23.83s for LLaMA 3 70B. Query rewriting adds minimal overhead when paired with GPT-OSS 120B (0.49–0.71s) but introduces measurable overhead with LLaMA 3 70B (9–10s), as the rewriting module invokes the same LLM backend — a coupling effect made visible only through component-level instrumentation.

TABLE IV
PIPELINE LATENCY ANALYSIS – MEDICAL DOMAIN (AVG. OVER 25 QUERIES, IN SECONDS)

Embedding	LLM	Embed (s)	Ret. (s)	Rew. (s)	LLM (s)	Total (s)
<i>Without Query Rewriting</i>						
BGE Large	GPT-OSS 120B	4.38	0.17	—	2.27	7.43
BGE Large	LLaMA 3 70B	4.03	0.18	—	14.00	18.96
MPNet Base v2	GPT-OSS 120B	3.29	0.18	—	2.41	6.50
MPNet Base v2	LLaMA 3 70B	2.37	0.11	—	23.83	26.83
<i>With Query Rewriting</i>						
BGE Large	GPT-OSS 120B	4.25	0.17	0.71	1.94	7.86
BGE Large	LLaMA 3 70B	6.34	0.21	9.22	17.60	34.14
MPNet Base v2	GPT-OSS 120B	2.80	0.13	0.49	2.04	6.03
MPNet Base v2	LLaMA 3 70B	2.53	0.08	9.52	19.83	32.55

Together, Tables I, II, III and IV illustrate how the proposed framework supports simultaneous evaluation of retrieval quality, generation faithfulness, pipeline latency and session-aware interaction — within a single configurable infrastructure and without requiring separate instrumentation tooling. To the best of our knowledge, no existing RAG framework supports deployment-ready stateful interaction, component-level latency profiling and multi-turn session evaluation in addition to component-level experimentation. The primary objective of this evaluation is not exhaustive benchmarking but to demonstrate that the framework natively has the capability for end-to-end experimentation across multiple dimensions.

V. DISCUSSION AND IMPLICATIONS

The experimental results provide several important insights regarding the design and deployment of modular RAG systems across heterogeneous domains.

1) Domain-Sensitive Retrieval Behavior: Retrieval performance varied across domains, reflecting differences in linguistic structure and knowledge organization. The Sanskrit corpus, characterized by conceptual abstraction and thematic recurrence, exhibited greater sensitivity to embedding selection. In contrast, the medical corpus, with structured terminology and well-defined biomedical phrasing, demonstrated more stable retrieval performance across embedding configurations. This observation underscores the importance of domain-aware

embedding evaluation when deploying RAG systems in diverse application settings.

2) Retrieval Quality and Generation Faithfulness: Higher Recall@5 and MRR values generally corresponded with improved faithfulness in generated responses. This supports the architectural principle that retrieval quality directly influences generation grounding. However, differences in relevance and correctness across LLM backends indicate that generation behavior is not solely determined by retrieval effectiveness. The modular separation of retrieval and inference layers enables controlled experimentation to study these interactions independently.

3) Architectural Modularity as an Enabler of Systematic Benchmarking: A key contribution of the proposed framework lies in its configuration-driven modularity. By isolating embedding models, retrieval strategies and LLM backends behind well-defined service interfaces, the system supports structured ablation studies without architectural restructuring. This capability is particularly valuable in rapidly evolving LLM ecosystems, where component substitution is frequent.

4) System-Level Capabilities: Beyond pipeline modularity, the empirical evaluation demonstrates that system-level capabilities — session management and latency instrumentation — contribute meaningfully to both deployment readiness and research utility. Multi-turn session evaluation reveals that maintaining conversational context meaningfully improves response relevance across turns, with relevance degrading progressively in the absence of session history. Component-level latency analysis demonstrates that embedding model selection and LLM backend choice carry measurable latency costs that practitioners must weigh alongside quality metrics. Critically, both findings are surfaced through the same unified framework without requiring separate tooling — validating the architecture’s design principle that system-level and research-level capabilities need not be mutually exclusive.

5) Toward Domain-Invariant Configuration Principles: A longer-term implication of this work concerns the feasibility of universal RAG configuration principles. Systematic experimentation across multiple domains using the proposed framework may reveal whether configuration patterns generalize — for instance, whether specific embedding families outperform others on abstractive versus structured corpora, or whether particular LLM backends demonstrate advantages across retrieval quality thresholds. The framework’s modular and configuration-driven design naturally supports such cross-domain experimentation, providing a foundation for this line of inquiry.

VI. CONCLUSION

We presented a service-oriented retrieval-augmented generation (RAG) architecture that unifies configuration-driven modularity, deployment-ready system services and component-level observability within a single framework. The architecture enables controlled experimentation across retrieval, ranking

and generation components while supporting practical deployment requirements such as persistent storage, session-aware interaction and role-based access control.

Using this framework, we evaluated RAG behavior across two heterogeneous domains: a Sanskrit philosophical corpus and a biomedical abstract corpus. The results demonstrated that identical RAG configurations can exhibit substantially different behavior depending on domain characteristics. While generation remained faithful to retrieved context in the Sanskrit domain, substantially lower faithfulness in the medical domain revealed a tendency for models to rely on parametric knowledge. Multi-turn evaluation further showed that maintaining conversational state improves response relevance, while latency instrumentation exposed performance interactions that are difficult to identify through quality metrics alone. These findings suggest that effective RAG deployment cannot be treated as a purely model-selection problem. Instead, retrieval strategies, system state and performance characteristics must be considered jointly, and configurations that perform well in one domain may not generalize reliably to another. By providing such a framework, this work offers a foundation for systematic investigation of domain-adaptive RAG systems. Future work will extend the framework to access-aware retrieval, knowledge-graph integration, non-linear execution pipelines and broader evaluation across additional domains and expanded query sets.

REFERENCES

- [1] X. Zhou, J. He, W. Zhou, H. Chen, Z. Tang, H. Zhao, X. Tong, G. Li, Y. Chen, J. Zhou *et al.*, “A survey of LLM $\times$ Data,” *arXiv preprint arXiv:2505.18458*, 2025.
- [2] Databricks, “Mosaic AI: Building Generative AI Applications,” 2024, accessed: March 2026. [Online]. Available: <https://www.databricks.com/product/artificial-intelligence>
- [3] Snowflake, “Snowflake Cortex AI,” 2025, accessed: March 2026. [Online]. Available: <https://www.snowflake.com/en/product/features/cortex/>
- [4] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen *et al.*, “Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models,” *Computational Linguistics*, vol. 51, no. 4, pp. 1373–1418, 2025.
- [5] M. A. K. Raiaan, M. S. H. Mukta, K. Fatema, N. M. Fahad, S. Sakib, M. M. J. Mim, J. Ahmad, M. E. Ali, and S. Azam, “A review on large language models: Architectures, applications, taxonomies, open issues and challenges,” *IEEE access*, vol. 12, pp. 26 839–26 874, 2024.
- [6] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, “Query rewriting in retrieval-augmented large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 5303–5315.
- [8] Y. Zhou, Q. Luo, B. Feng, and B. Wang, “Large language models for reranking: A survey,” *Authorea Preprints*, 2025.
- [9] F. Xu, W. Shi, and E. Choi, “Recomp: Improving retrieval-augmented lms with compression and selective augmentation,” *arXiv preprint arXiv:2310.04408*, 2023.
- [10] S. Siriwardhana, R. Weerasekera, E. Wen, T. Kaluarachchi, R. Rana, and S. Nanayakkara, “Improving the domain adaptation of retrieval augmented generation (RAG) models for open domain question answering,” *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1–17, 2023.
- [11] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, “Repocoder: Repository-level code completion through iterative retrieval and generation,” in *Proceedings of the 2023*

Conference on Empirical Methods in Natural Language Processing, 2023, pp. 2471–2484.

- [12] Y. Ding, Z. Wang, W. Ahmad, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, “Cocomic: Code completion by jointly modeling in-file and cross-file context,” in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 2024, pp. 3433–3445.
- [13] M. Arslan, H. Ghanem, S. Munawar, and C. Cruz, “A survey on RAG with LLMs,” *Procedia Computer Science*, vol. 246, pp. 3781–3790, 2024.
- [14] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, H. Wang *et al.*, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, vol. 2, no. 1, p. 32, 2023.
- [15] LangChain AI, “GitHub - langchain-ai/langchain: The agent engineering platform — github.com,” <https://github.com/langchain-ai/langchain>, 2026, [Accessed 28-03-2026].
- [16] LlamaIndex, “GitHub - run-llama/llama_index: LlamaIndex is the leading document agent and OCR platform — github.com,” https://github.com/run-llama/llama_index, 2026, [Accessed 28-03-2026].
- [17] X. Zhang, Y.-Z. Song, Y. Wang, S. Tang, X. Li, Z. Zeng, Z. Wu, W. Ye, W. Xu, Y. Zhang *et al.*, “Raglab: A modular and research-oriented unified framework for retrieval-augmented generation,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2024, pp. 408–418.
- [18] Y. Gao, Y. Xiong, M. Wang, and H. Wang, “Modular rag: Transforming rag systems into lego-like reconfigurable frameworks,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21059>
- [19] J. Jin, Y. Zhu, Z. Dou, G. Dong, X. Yang, C. Zhang, T. Zhao, Z. Yang, and J.-R. Wen, “Flashrag: A modular toolkit for efficient retrieval-augmented generation research,” in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 737–740.
- [20] Y. Chen, D. Guo, S. Mei, X. Li, H. Chen, Y. Li, Y. Wang, C. Tang, R. Wang, D. Wu *et al.*, “Ultrarag: A modular and automated toolkit for adaptive retrieval-augmented generation,” *arXiv preprint arXiv:2504.08761*, 2025.
- [21] T. Zhang, S. G. Patil, N. Jain, S. Shen, M. Zaharia, I. Stoica, and J. E. Gonzalez, “Raft: Adapting language model to domain specific rag,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.10131>
- [22] Q. Jin, B. Dhingra, Z. Liu, W. W. Cohen, and X. Lu, “Pubmedqa: A dataset for biomedical research question answering,” *CoRR*, vol. abs/1909.06146, 2019. [Online]. Available: <http://arxiv.org/abs/1909.06146>
- [23] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.
- [24] S. Es, J. James, L. E. Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” in *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations*, 2024, pp. 150–158.
- [25] Google, “Gemini 2.5 Flash,” <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-flash>, 2025, accessed: March 2026.
- [26] Meta, “Introducing Meta Llama 3,” <https://ai.meta.com/blog/meta-llama-3/>, 2024, accessed: March 2026.
- [27] OpenAI, “Introducing GPT-OSS,” <https://openai.com/index/introducing-gpt-oss/>, 2025, accessed: March 2026.
- [28] BAAI, “BGE: BAAI General Embedding,” <https://huggingface.co/BAAI/bge-large-en>, 2023, accessed: March 2026.
- [29] R. Xu, Z. Qi, Z. Guo, C. Wang, H. Wang, Y. Zhang, and W. Xu, “Knowledge conflicts for LLMs: A survey,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 8541–8565. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.486/>
- [30] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl *et al.*, “Large language models encode clinical knowledge,” *Nature*, vol. 620, no. 7972, pp. 172–180, 2023.